

Task1

Prompt1: “How can I decompile an Android APK on macOS?”

Outcome1: ChatGPT recommended using JADX or Apktool for APK decompilation on macOS and suggested JADX as the simpler choice for this task. It provided step-by-step setup guidance, including installing Homebrew, installing JADX, and opening the APK in the JADX GUI for source-code inspection.

Task2

Prompt 1: I am doing INFO5995 Assignment 1 Task 2. Help me write a 3–5 sentence summary of an Android app with registration, login, session token storage, and backend authentication. Use simple clear language.

Outcome 1: This Android application provides user authentication functions including registration, login, and logout. It stores user credentials and session tokens locally on the device to maintain login status. The app communicates with a remote backend server to verify user identity and grant access to protected content. Session tokens are the core asset used for authorization after successful login.

Prompt 2: For INFO5995 Assignment 1 Task 2, create a simple system model for an Android auth app. Include components, key assets, and data flows. Link it to cryptographic/randomness vulnerability analysis.

Outcome 2: System components: UI (Login/Register Activities), Local Storage (SharedPreferences), Network Channel, Backend Server.

Key assets: username, password, session token, auth state.

Data flow: User input → UI → Local Storage (token save) → Network → Server validation.

This model directly supports later analysis of token generation (randomness) and transmission (cryptography).

Prompt 3: Based on INFO5995 Assignment 1 rubric, give realistic assumptions and attacker goals for threat modeling linked to weak randomness/crypto vulnerabilities.

Outcome 3: Core Assumptions: Default device is non-rooted. Tokens are generated client-side for login sessions. Tokens are sent to the server for authorization.

Attacker Goals: Predict or steal session tokens. Impersonate valid users. Bypass login authentication. Exploit weak randomness to forge valid tokens.

Prompt 4: You are a TA grading INFO5995 Assignment 1. Use this rubric to ask 4 targeted questions about my Task 2 (System & Threat Model) work.

Rubric for System & Threat Model (4 marks):

- 4 marks: clear system model, key assets/data flows, realistic attacker goals/capabilities **explicitly linked to the chosen vulnerability**.
- 3 marks: main components/assets and at least one plausible attacker described; mostly consistent.

- 1–2 marks: partial model, vague assumptions, weak linkage.
- 0 marks: missing or wrong.

My Task 2 work:

1. App summary: This app handles user registration, login, and session management using client-generated tokens stored in SharedPreferences.
2. System model: UI → Local Storage → Network → Server; assets = token, password.
3. Assumptions: non-rooted device; tokens used for auth.
4. Attacker goal: predict weak random tokens to impersonate users.

Outcome 4: Does your system model clearly show where the session token is generated and why it links to randomness/crypto vulnerability?

What evidence from the APK supports that session token is a key asset?

Why are your attacker goals realistic for this app's vulnerability?

How does your threat model support the later vulnerability explanation in Task 3?

Answer: My model marks token generation at the client app layer, which is exactly where weak randomness occurs. This directly links to the crypto/randomness scope of Assignment 1.

From decompiled LoginActivity, the app uses a token to maintain login state and stores it in SharedPreferences, so it is a critical security asset.

The attacker can predict tokens if randomness is weak, which allows account takeover — a realistic impact for auth apps.

My threat model defines what the attacker can do, which I use to build the step-by-step attack scenario in Task 4.

Task 3

Prompt 1:

Help me identify all randomness-related values in the decompiled APK and determine which ones may be security-relevant.

Outcome 1:

Two randomness-related locations were identified through static analysis:

MainActivity.randomNumberGenerator(), which generates an integer between 0 and

99 using `java.util.Random`, and `Login.generateSessionToken()`, which generates a 16-character alphanumeric string using `java.util.Random`.

Prompt 2:

Help me classify the security role of each random value and decide whether both should be treated as vulnerabilities.

Outcome 2:

`MainActivity.randomNumberGenerator()` was classified as a general-purpose or UI/demo-related random value because it is not clearly used in authentication, authorization, or session control. `Login.generateSessionToken()` was classified as security-sensitive because its output is used as a session token and stored through `createSession()` in `KEY_SESSION_TOKEN`.

Prompt 3:

Help me verify the code flow for the session token and confirm whether this random value is actually involved in the login process.

Outcome 3:

Static analysis confirmed that after successful login, `createSession()` is called. This method stores the output of `generateSessionToken()` in `KEY_SESSION_TOKEN`. This established the code chain: successful login -> `createSession()` -> `generateSessionToken()` -> token stored as session data.

Prompt 4:

Help me determine whether the app contains an explicit token validation or authorization branch after the token is created.

Outcome 4:

A follow-up code search did not identify a clear explicit token validation branch such as `if (token != null)` or similar authorization logic. As a result, the confirmed finding was narrowed to weak randomness used for session token generation, without overstating a full session bypass claim.

Prompt 5:

Help me explain why `java.util.Random` is inappropriate in this case while not every use of `Random` is a vulnerability.

Outcome 5:

The analysis concluded that not all uses of `Random` are security issues. The key factor is whether the generated value has a security role. Since the value in `MainActivity` is not security-sensitive, it was not selected as the core issue. In contrast, the value in `Login.generateSessionToken()` is used as a session credential, so using `java.util.Random` here is inappropriate because it reduces token unpredictability.

Prompt 6:

Help me evaluate our Task 3 analysis against the rubric and ask follow-up questions about the randomness-related findings, security role classification, and vulnerability selection.

Outcome 6:

The mock Q&A confirmed that our Task 3 evidence was strongest around `Login.generateSessionToken()`. It also helped us make the distinction clearer

between a general-purpose random value in `MainActivity` and a security-sensitive session token in `Login`. Based on this review, we refined the wording of our Task 3 section and made the core vulnerability selection more explicit.

LLM mock Q&A:

Question1: How did you identify the randomness-related locations in the APK?

Answer1: We used static analysis on the decompiled APK and searched for randomness-related APIs. This identified two uses of `java.util.Random` in `MainActivity` and `Login`.

Question2: Why is `MainActivity.randomNumberGenerator()` not the main vulnerability?

Answer2: It only generates a normal integer and is not clearly related to login, authorization, or session control.

Question3: Why is `Login.generateSessionToken()` security-sensitive?

Answer3: It generates a value that is stored as `KEY_SESSION_TOKEN` after login, so it functions as a session token.

Question4: Why is `java.util.Random` inappropriate here?

Answer4: A session token requires strong unpredictability, but `java.util.Random` is not designed for security-sensitive use.

Question5: What is the core vulnerability in Task 3?

Answer5: The core vulnerability is weak session token generation in `Login.generateSessionToken()`.

Question6: What evidence supports your conclusion?

Answer6: The evidence comes from the code chain identified through static analysis. `Login.generateSessionToken()` uses `java.util.Random` to create a token, and `createSession()` stores that token in `KEY_SESSION_TOKEN` after successful login. This shows that the generated value is both session-related and security-sensitive. This review led us to revise the report by making the security role of `Login.generateSessionToken()` clearer and by avoiding unsupported claims about full session bypass.

Question7: What fix would you recommend?

Answer7: Replace `java.util.Random` with `java.security.SecureRandom`. `SecureRandom` is more suitable because it provides stronger unpredictability for security-sensitive values.

AI Recommend Tool & References:

- JADX
- OWASP MASTG-TEST-0204: Insecure Random API Usage

Task 4

Prompt 1:

Explain where the value is generated, why the method is unsuitable for its security role, and what attacker knowledge/observation is needed to exploit it.

Outcome 1:

The vulnerability originates from the method `generateSessionToken()` in the `Login.java` class, where a 16-character alphanumeric session token is generated using `java.util.Random`. This token is subsequently stored via `createSession()` in `SharedPreferences` and serves as a session identifier.

The core issue is that `java.util.Random` is a general-purpose pseudo-random number generator, not a cryptographically secure PRNG (CSPRNG). Its outputs are predictable and deterministic, reducing token entropy and making it unsuitable for security-sensitive credentials.

An attacker does not need access to the source code but must be able to observe, repeat, or collect multiple token samples generated over time to exploit this vulnerability.

Prompt 2:

Describe a realistic attacker model (e.g., network observer, another app, rooted-device user) and concrete capabilities.

Outcome 2:

Realistic attacker model includes low-privilege but capable adversaries:

- Attacker Types:
 1. Network adversary (e.g., Man-in-the-Middle in the victim's local network).
 2. Malicious co-resident application installed on the victim's device (no root required).
- Attacker Capabilities:
 1. Zero-privilege execution: attacker does not require physical access or root.
 2. Dynamic sample collection: attacker can intercept network traffic or repeatedly trigger login to collect session token samples.
 3. Offline analysis: attacker can use local computation or scripts to analyze collected tokens, leveraging the predictability of `java.util.Random`'s linear congruential generator to infer future tokens.

Prompt 3:

Provide a short step-by-step attack scenario (for example, prediction/influence leading to auth bypass or impersonation).

Outcome 3:

Step-by-step attack scenario:

1. Observation and sample collection: Attacker collects multiple 16-character session tokens by triggering login or monitoring traffic.

2. PRNG analysis: Using the predictability of `java.util.Random`, the attacker analyzes collected tokens to infer the internal state of the PRNG.
3. Token prediction: Attacker predicts future session tokens with higher probability than random guessing.
4. Session impersonation: Attacker injects predicted or previously observed tokens into requests or replaces local tokens. The system accepts these forged tokens, allowing unauthorized session access and account impersonation.

AI Log – Rubric-driven Mock Q&A (Task 4)

Q1: Where is the value generated and why is it insecure?

A: The session token is generated in `Login.generateSessionToken()` using `java.util.Random`, which is deterministic and not suitable for security-sensitive tokens.

Q2: What attacker capabilities are required?

A: The attacker can collect multiple tokens via network observation or repeated login attempts and perform offline analysis.

Q3: Why is the attacker model realistic?

A: It includes common attackers such as MITM on public Wi-Fi or malicious apps without root privileges.

Q4: What is the security impact?

A: The attacker can predict or reuse tokens to impersonate users and bypass authentication.

Q5: What evidence supports this?

A: Static analysis with JADX shows the use of `java.util.Random` in token generation.

Improvements:

- Added discovery method (JADX)
- Clarified authentication bypass impact